

KEYFORTA

Solution Architecture Overview

Kinshasa-first property discovery and rental-management platform

Agenda

- 1 Product & architecture principles
- 2 System landscape — apps, packages, and boundaries
- 3 Deployment architecture on Azure
- 4 Data, tenancy, and financial integrity
- 5 Security, authorization, and AI safety boundaries
- 6 Delivery pipeline and quality gates
- 7 Key architecture decisions (ADRs)
- 8 Open decisions before production launch

Product Invariants Drive the Architecture

Financial integrity

Money is never binary floating point; posted transactions are reversed and replaced, never silently edited.

Zero-trust authorization

Every organization, user, role, property, unit, and lease ID is re-validated at the API boundary — never trusted from the client.

AI stays narrow

AI models never touch the database directly; they call narrow, typed, authorized tools only.

Humans decide

Acceptance, eviction, legal, pricing, refund, and money-movement decisions remain human, not autonomous.

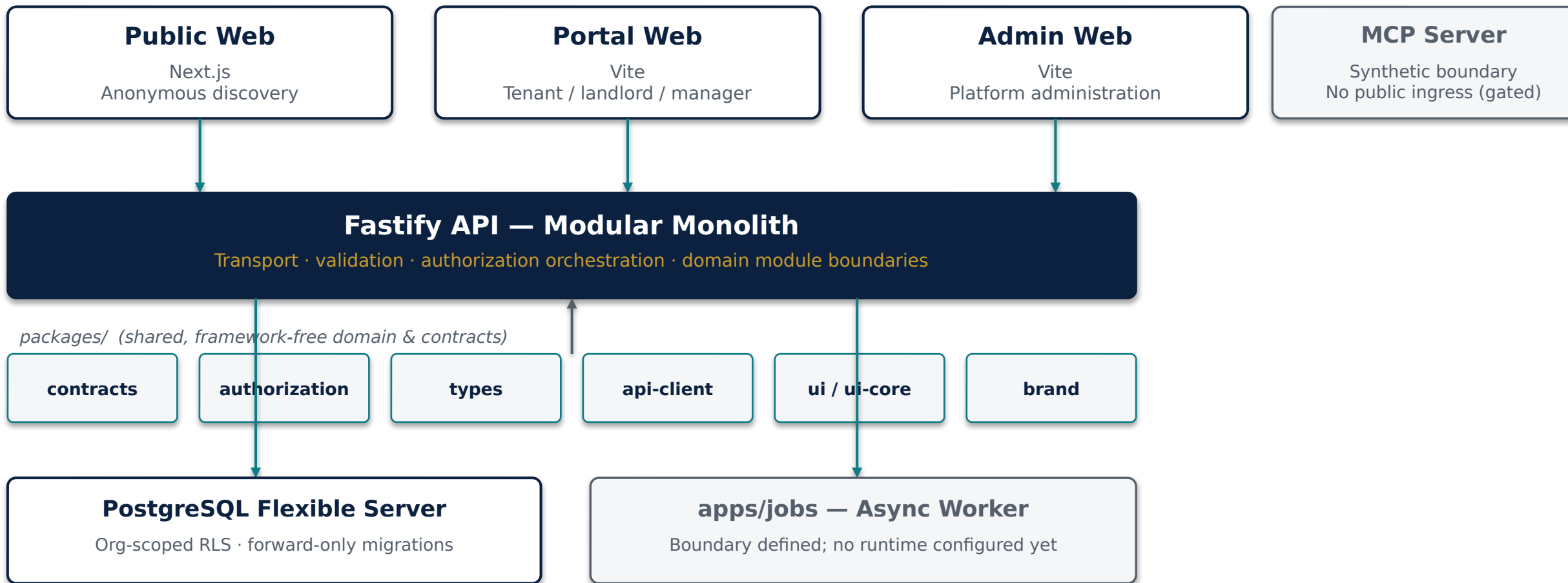
Evidence & audit

Document versions, approvals, and audit correlation IDs are preserved end to end.

Resilient by design

Core workflows keep working even when AI services are unavailable.

Apps, Shared Packages & Boundaries



Browsers call the public API directly (no BFF hop); every app consumes shared contracts instead of duplicating financial or authorization logic.

Azure Runtime — Lean Single-Environment Pilot

Resource Group — dev (South Africa North)

Container Apps Environment

Public Web

public HTTPS ingress

Portal / Admin Web

public HTTPS ingress

Fastify API

public HTTPS ingress (direct browser access)

PostgreSQL Flexible Server

Burstable SKU · 7-day backups
Managed-identity auth (Entra)

Container Registry (Basic)

Log Analytics (30-day retention)

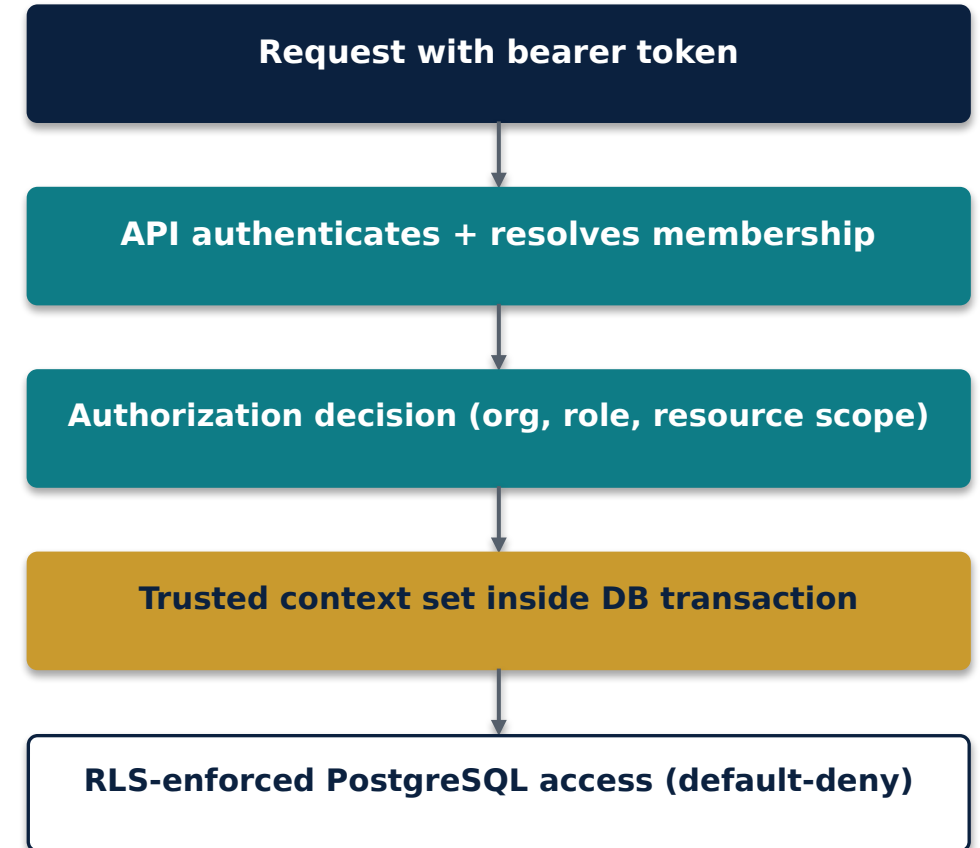
Narrowly-scoped managed identities: deployment · web · API · migration

No long-lived Azure credentials — GitHub OIDC federates per-environment identities.

Deferred until justified by demand: test/prod stamps, Service Bus, Blob Storage, Key Vault, Application Insights, general-purpose workers, private endpoints, HA, geo-redundant backups.

Organization Isolation and Financial Integrity

- Every organization-owned table carries a non-null `organization_id`.
- The API resolves membership and authorizes every command/query before touching persistence — client IDs are never trusted.
- PostgreSQL row-level security defaults to deny when trusted organization context is absent (defense in depth).
- The runtime role cannot own or bypass RLS on protected tables; `FORCE ROW LEVEL SECURITY` closes ownership loopholes.
- A separate migration identity owns schema change; application startup never auto-applies migrations.



**Money uses integer minor units, never binary floating point.
Posted financial transactions are reversed and replaced — never silently edited.**

Authorization Boundaries and Human Control

Application-boundary authorization

No organization, user, role, property, unit, or lease ID from a client is ever trusted without re-validation at the API.

AI never touches the database

Models call narrow, typed, authorized tools only — never direct SQL or persistence access.

Human-only decisions

Tenant acceptance, eviction, legal, pricing, refunds, and money movement are never autonomous.

Evidence & audit trail

Document versions, evidence references, human approvals, and audit correlation IDs are preserved end to end.

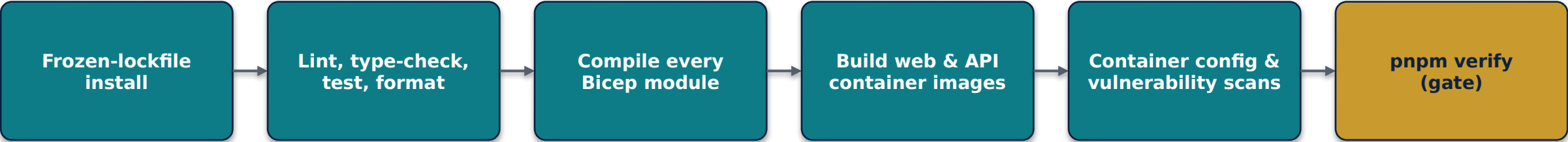
Fail-safe without AI

Core workflows (e.g., schedule preview) keep working when AI services are unavailable.

Least-privilege identities

Separate managed identities for web, API, worker, database, and deployment — no shared credentials.

CI/CD Pipeline and Quality Gates



GitHub Actions with OIDC — no long-lived Azure credentials

Identity	Per-environment federated deployment identity via workload identity federation
Environments	dev / test / production GitHub Environments; manual approval required for production
Promotion	Deployments allowed only from the protected main branch, tied to an immutable commit SHA
Evidence	CI worktree must stay clean after build; evidence-manifest gate fails otherwise

Key Architecture Decision Records

ADR	Decision	Why it matters
ADR-0001	Modular monolith	One API, internally enforced domain modules; service extraction deferred until measurable value
ADR-0002	TypeScript monorepo	pnpm + Turborepo; framework-free domain code kept portable
ADR-0004	PostgreSQL + app authorization + RLS	Application authorizes first; RLS is defense in depth, not the authorizer
ADR-0006	Lean single-environment pilot	One dev environment; test/prod and extra services deferred until justified
ADR-0008	Direct browser → API access	Removed the web BFF hop; API independently authorizes every request
ADR-0005	GitHub OIDC deployment	No long-lived Azure credentials; federated identity per environment

Open Decisions Before Production Launch

Decision needed	Required before
Identity provider tenant & onboarding	Protected production access
PostgreSQL hosting region	Production personal data
Payment provider	Automated payments
Object storage & malware scanning	Document uploads
Messaging provider (email/SMS/WhatsApp)	Outbound communications
DRC retention / lease / notice policy	Commercial leasing
Backup & recovery targets (RPO/RTO)	Production launch
Observability & on-call ownership	Production launch

Owners: Technology, Finance, Product, Privacy. Each decision requires context, options, selected choice, security/privacy and cost impact, and documented approval evidence.

Questions & Discussion

[docs/adr](#) · [docs/architecture/adr](#) · [docs/contracts](#) — full authoritative detail